

Flowdesk AI – Product & Engineering Specification

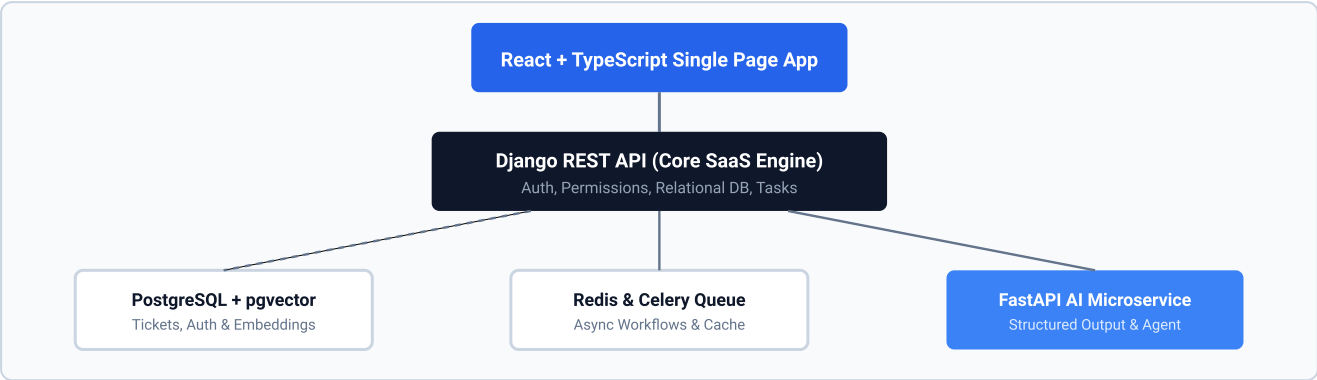
Full-Stack AI Product Operations Platform Architecture & Portfolio Roadmap

CORE STACK	AI CAPABILITY	DATABASE	DEPLOYMENT
React / DRF / FastAPI	RAG / Agent Tools / Structured	PostgreSQL + pgvector	Docker / Redis / Async

Executive Overview: Flowdesk AI is an enterprise-grade AI Product Operations platform designed to demonstrate full-stack SaaS engineering. Unlike basic wrappers, Flowdesk integrates an AI layer into operational workflows: automated structured issue classification, agentic tool invocation, vector-based knowledge retrieval, and deterministic UI rendering driven by LLM structured outputs.

1. High-Level Architecture & Ecosystem

The system leverages a decoupled microservices-ready architecture. Django REST Framework serves as the primary relational data store and API engine, while a dedicated FastAPI service handles asynchronous AI tasks, vector embeddings, and agent orchestration.



2. Target Skill Alignment Matrix

Domain	Technologies & Standards	Flowdesk Implementation Scope
Frontend	React 18, TypeScript, Vite, Tailwind/CSS	Responsive dashboard, optimistic UI updates, design system, analytical charts.
Backend API	Python 3.11+, Django REST Framework, DRF Auth	RESTful endpoints, object permissions, concurrency control, custom serializers.
AI Service	FastAPI, OpenAI API, Pydantic, RAG	Structured Outputs, tool function calling execution loop, similarity search.
Infrastructure	Docker, PostgreSQL, Redis, Celery	Containerized multi-service setup, background queue for AI pipelines.

3. Key System Capabilities

1. Core SaaS & Ticket Engine

DRF Core

Full lifecycle management for tickets: state transitions (*New* → *Investigating* → *Waiting* → *Resolved*), assignment, tagging, metadata attachments, optimistic UI, and row-level locking for concurrency control.

2. AI Ticket Classification

Pydantic

On creation, tickets trigger an async FastAPI pipeline returning strict JSON schema results: category, priority rating, customer sentiment, summary, and suggested tags, validated before DB persistence.

3. Knowledge Base & RAG

pgvector

Documentation search powered by text embeddings. User queries undergo vector similarity search against indexed technical docs, generating verified answers with cited source articles.

4. Autonomous AI Agent

Tool Calling

Agent capable of parsing natural language requests (e.g. *"Find unresolved export bugs"*), selecting and executing registered functions, and taking actions back in Django.

4. Dashboard UI Design Blueprint

The main interface demonstrates clean UI engineering, balance, and clear information hierarchy:

FLOWDESK AI // OPERATIONS DASHBOARD

🔔 Notifications | Profile: Koryun

NAV

- Dashboard
- Tickets
- Customers
- Knowledge
- AI Agent
- Settings

128

OPEN TICKETS

47

AI RESOLVED

18m

AVG RESPONSE

TICKET VOLUME & INCIDENT TRENDS

[Trend Chart Line: Peak on Export Module Issues]

5. AI Integration Mechanics: Structured Outputs & Tools

To ensure deterministic system behavior, LLMs return payload objects mapped directly to Pydantic models on the backend and TypeScript interfaces on the React client.

```
// AI Ticket Analysis Schema Validation (FastAPI / Pydantic) class
TicketAnalysisResult(BaseModel): category: Literal["bug", "feature_request", "billing",
"general_inquiry"] priority: Literal["low", "medium", "high", "urgent"] sentiment:
Literal["positive", "neutral", "negative"] summary: str = Field(description="1-2 sentence overview
of the issue") suggested_tags: List[str] suggested_response: str // Agent Tool Execution Suite
REGISTERED_TOOLS = [ "search_tickets", "get_customer_profile", "search_knowledge_base",
"update_ticket_status" ]
```

6. Phased Implementation Roadmap

A structured six-phase engineering plan ensures incremental delivery of core SaaS infrastructure before layering complex AI capabilities.

Phase	Focus Area	Key Deliverables
Phase 1	Core SaaS Foundation	React + Vite scaffolding, DRF setup, JWT auth, database schemas for Tickets, Customers, and Comments.
Phase 2	Engineering Quality	Pytest & React Testing Library suite, OpenAPI doc generation, optimistic updates, pagination & filtering middleware.
Phase 3	AI Classification	FastAPI service integration, structured JSON validation, automatic metadata generation on incoming tickets.
Phase 4	RAG & Knowledge Base	Vector embedding pipeline (pgvector), article management, similarity search, and source-cited answer generation.
Phase 5	Tool-Using AI Agent	Function calling agent loop, multi-tool orchestration (<code>`search_tickets`</code> , <code>`update_ticket`</code>), natural language console.
Phase 6	Production & Hardening	Docker Compose multi-container orchestrations, CI/CD pipelines, rate limiting, structured logging & monitoring.

7. Portfolio Positioning & Interview Talking Points

"What have you built with Django?"

"I built the backend for Flowdesk AI, a multi-tenant SaaS platform utilizing Django REST Framework. It features JWT authentication, relational model design for ticket operations, custom permission classes, optimistic concurrency handling, PostgreSQL persistence, and background task execution via Celery and Redis."

"How do you approach real-world AI integration?"

"Instead of adding an unconstrained chatbot, I integrated AI as an operational layer. I built a FastAPI microservice utilizing OpenAI Structured Outputs validated via Pydantic to feed deterministic React UIs, implemented a pgvector RAG pipeline for documentation retrieval, and designed a function-calling agent capable of safely executing database actions."

Flowdesk AI validates proficiency across full-stack engineering, modern UI design systems, asynchronous microservices, and reliable AI systems architecture.